

Собственная функция натурального логарифма vogf (Vova's logarithm)

Владимир Попов

1 апреля 2026 г.

Содержание

1	Аннотация	3
2	Написание vogf	4
2.1	Базовая версия	4
2.2	Lookup Table	4
3	Тестирование	4

1 Аннотация

Целью является написание собственной функции вычисления натурального логарифма, без зависимостей от сторонних библиотек, предоставляющих аналогичный функционал. Входной и выходной аргументы будут типа `float`. Язык написания - Си. Точность результата - 3.5 ULP.

Флаги и коды ошибок должны выставляться аналогично функции `logf` из библиотеки `libm`. Проверка точности будет производиться при помощи `mpfr`. Также для параллельного тестирования будет использоваться OpenMP. Система сборки - Make.

Параметр	Значение
<i>Аппаратное обеспечение (Hardware)</i>	
Процессор (CPU)	AMD Ryzen 7 8845H (8 ядер, 16 потоков)
Архитектура	x86_64
Базовая / Максимальная частота	3.8 ГГц / 5.1 ГГц
L1 Кэш (данные/инструкции)	256 / 256 КиБ
L2 Кэш	8 МиБ
L3 Кэш	16 МиБ (общий)
Оперативная память	32 ГБ LPDDR5x
<i>Программное обеспечение (Software)</i>	
Операционная система	Ubuntu 22.04.5 LTS
Ядро Linux	6.8.0-106-generic
Компилятор	GCC 11.4 / Clang 11.4
Библиотека C	glibc 2.35
Математические библиотеки	libm, MPFR (для верификации)
Параллелизация	OpenMP 4.5+
Флаги компиляции	-DNDEBUG -O2 -fopenmp

Таблица 1: Характеристики тестовой системы и программного окружения

2 Написание `vogf`

2.1 Базовая версия

Напишем обычную аппроксимацию логарифма при помощи заранее предподсчитанных коэффициентов минимаксного полинома. Для значений коэффициентов будем использовать программу `sollya`.

```
1 $ sollya
2 > fpminimax(log1p(x), [|1,2,3,4,5,6|], [|single, single, single, single, single,
   single|], [0, 1]);
3 x * (0.999991595745086669921875 + x * (-0.499361336231231689453125 + x *
   (0.325206577777862548828125 + x * (-0.21004854142665863037109375 + x *
   (0.10122220218181610107421875 + x * (-2.386914193630218505859375e-2))))))
```

Делаем редукцию мантиссы к полуинтервалу $[1, 2)$, вычитаем единицу, раскладываем функцию $\ln(x + 1)$ при помощи полученной аппроксимации и добавляем экспоненциальную часть $\text{row} \cdot \ln(2)$. Если машина поддерживает FMA инструкции, то компилятор преобразует умножение $+$ сложение в них при вычислении значения полинома в точке, благодаря расстановке скобок.

2.2 Lookup Table

Вычислять ряд до 6 порядка это долго. Мы можем предподсчитать значения для первых k старших битов (с занулёнными младшими) и раскладывать в полином только остаток.

$$\ln(\text{mant}) = \ln(\text{mant} \cdot \frac{R_i}{R_i}) = \ln(\text{mant} \cdot R_i) - \ln(R_i) \quad (1)$$

, где $R_i = 1.0 + \frac{i+0.5}{2^k}$ - среднее число в i -ом интервале.

Обозначим $T_i = -\ln(R_i)$, Это будет вторая таблица предподсчитанных значений. Напишем скрипт, который будет генерировать эти таблицы для произвольных значений k .

Итоговая версия функции

3 Тестирование

Было написано 3 функции для тестирования

1. Проверка результатов вычисления аргументов из массива. Эталон - `mpfr` с точностью 128 бит

2. Проверка результатов вычисления всех аргументов от 0 до INFINITY. Эталон - mpfr с точностью 128 бит. Вычисление выполняются параллельно при помощи OpenMP.
3. Проверка выставления флагов. Эталон - logf из libm.

Тип входа	Аргумент x	Результат <code>vogf</code>	Результат <code>mpfr</code>	Ошибка (ULP)
Отрицательное число	-1.0	nan	nan	0
nan	nan	nan	nan	0
Ноль	0.0	$-\infty$	$-\infty$	0
Бесконечность	$+\infty$	$+\infty$	$+\infty$	0
Единица	1.0	0.0	0.0	0
Мин. норм.	1.175494×10^{-38}	-87.33655	-87.33655	0
Макс. норм.	3.402823×10^{38}	88.72284	88.72284	0
Денормал.	1.401298×10^{-45}	-103.27893	-103.27893	0
Целое число	2.0	0.693147	0.693147	0
Число π	3.141593	1.144730	1.144730	0
Число e	2.718282	0.999999	0.999999	0
Худший случай	0.99999994	-5.960466×10^{-8}	-5.960464×10^{-8}	1.750

* Тестирование проведено на всём диапазоне положительных значений. Max ULP error: 1.750.

Таблица 2: Результаты тестирования точности функции `vogf`

Описание	<code>vogf</code>		<code>logf</code>	
	errno	fenv	errno	fenv
Отрицательное число	21 (EDOM)	1 (FE_INVALID)	21	1
Отрицательная бесконечность	21 (EDOM)	1 (FE_INVALID)	21	1
Положительный ноль	22 (ERANGE)	4 (FE_DIVBYZERO)	22	4
Отрицательный ноль	22 (ERANGE)	4 (FE_DIVBYZERO)	22	4
Положительная бесконечность	0	0	0	0
Неточное значение (Inexact)	0	20 (FE_INEXACT)	0	20

Таблица 3: Результаты тестирования исключительных ситуаций `vogf`

Проверка всех положительных чисел на моё машине заняла около 10 минут, я считаю это вполне приемлемо. Полный результат тестирования можно посмотреть здесь.

Наибольшая ошибка наблюдается на числах около 1, из-за катастрофической потери точности. Чтобы решить эту проблему без ухудшения времени вычислений добавлением больших порядков полинома, Таблица T хранилась в формате double и вычисления производились также в них.

По результатам тестирования эмпирически было выявлено, что для использования третьего порядка полинома достаточно $k = 8$, а для второго - $k = 12$. Если использовать таблицы с double размером 2^{12} , то на моей машине это будет занимать одну четвёртую часть кэша первого уровня. Если учитывать, что пользователь скорее всего сам будет полагаться на использование кэша (а также функции из других библиотек, которые он использует), то релевантнее будет использовать $k = 8$, чтобы не упереться в memory throughput. Тем более не у всех такие большие кэши.